Software Engineering for AI-Assisted Testing: A Test Intent Perspective

Yundong Shen

WASP Software Engineering Assignment, 2026

1 Introduction

My research starts from a practical problem in software testing. A test can compile, run, and pass, yet still express the wrong verification purpose. It may also express the right purpose so poorly that another developer cannot recover it. Test names, inputs, mocks, assertions, comments, framework conventions, and related project artifacts may each contain part of the explanation. When these parts do not provide a clear account of why a test exists, developers accumulate *intent debt*. They can see what the test executes, but they must infer which behaviour the test is meant to protect.

My ongoing work studies *Test Intent* as a structured representation of this information:

$$\langle \text{Test Objects},\ \text{Test Goal},\ \text{Test Activities} \rangle,$$

where the three elements state what is tested, why it is tested, and how the test performs the check. Evidence for these elements need not come only from test code. It may also come from a manual test description, a requirement, an issue, a review discussion, or direct input from a person. The level of detail depends on the task. An onboarding document needs a short explanation that is easy to read. Test generation may require exact preconditions, actions, and expected outcomes.

The current study first defines rules for extracting Test Intent and then evaluates zero-shot LLM extraction on 211 test cases from eight test suites and six testing categories. Its first downstream application is onboarding documentation, as the immediate aim is to help developers understand and review existing tests. If the extracted intent is sufficiently accurate and detailed, it could later support test generation, test improvement, retrieval, maintenance, and agent-based testing.

2 Two Principles from the Main Lectures

2.1 Verification, validation, and the test-oracle problem

The first concept is the distinction between verification and validation. Verification asks whether a system conforms to its specification, while validation asks whether the system meets its intended needs [1]. I find this distinction useful because it explains a common problem in testing. A test may run and pass because the implementation satisfies its assertion. However, the result does not show whether that assertion represents the behaviour that developers actually want to preserve. Determining the correct expected behaviour is known as the test-oracle problem [2].

Test Intent provides a practical way to examine this problem. When I read a test, I can separate the subject being tested, the purpose of the test, and the activities used to perform the check. The Test Object identifies the subject. The Test Goal records the verification purpose supported by the available evidence. The Test Activities describe how the test produces its result. This separation makes it easier to notice when the activities do not support the stated goal. It also reveals cases in which the goal itself cannot be

established from the available artifacts. In such cases, Test Intent should preserve the uncertainty instead of supplying an unsupported explanation.

This concept also affects how I evaluate Test Intent extraction. In the current study, an extraction is fully aligned only when its Test Objects, Test Goal, and Test Activities match the manually established Ground Truth. An important omission results in partial alignment, while an incorrect central goal results in misalignment. For future work on test generation, I would evaluate executability, coverage, and agreement with the intended behaviour separately.

2.2 Behavioural software engineering and bias-aware evaluation

The second idea is that software engineering decisions are influenced by human judgement and cognitive bias. Studies have identified several forms of cognitive bias in software engineering, including confirmation bias and anchoring [3]. These biases are relevant to Test Intent because its Ground Truth requires researchers to interpret evidence. An annotator may rely too heavily on the test name, treat the assertion as a complete description of the purpose, or keep the first interpretation that appears reasonable.

Working on the Ground Truth made this issue concrete for me. Three researchers reviewed all 211 cases, and the raw agreement was approximately 90%. When our labels differed, we discussed the evidence and agreed on a final interpretation. The discussion was useful because a disagreement could reveal either an annotation mistake or an unclear extraction rule. It also allowed us to compare different readings of the same test before deciding on the final label.

We did not calculate a chance-corrected measure such as Cohen's kappa or Krippendorff's alpha. Raw percentage agreement is easy to understand, but it does not account for agreement that may occur by chance [4]. I would therefore describe our result in a limited way: the researchers often produced the same initial interpretation, and the remaining differences were resolved through discussion. In a later study, I would retain the initial labels and record the evidence and reasoning used for each disputed case. This record would make the final decisions easier to inspect and would help us revise the extraction rules when similar disagreements appear again.

3 Two Principles from the Guest Lectures

3.1 Requirements engineering: separating why, what, and how

The first concept is to separate why a system is needed, what it should provide, and how it should be implemented. Requirements engineering identifies the purpose of a system and the needs of its stakeholders. It then documents these needs so that they can be analysed, discussed, and implemented [5]. This separation helps prevent an implementation decision from being treated as a requirement.

This concept provides a useful structure for Test Intent. The Test Objects describe what is examined, the Test Goal explains why the check exists, and the Test Activities describe how it is performed. The mapping is not exact because a goal extracted from a test may only represent its local verification purpose. It may not explain the broader user or business need. Test Intent can therefore help connect tests with requirements and identify differences between the purpose expressed by a test and the purpose stated in its related requirement.

3.2 Human aspects of AI-mediated software work

The second idea is to understand AI-assisted development as a workflow involving both people and tools. Studies of code-generating assistants show that programmers use them to accelerate familiar work and explore possible solutions [6]. Effective human–AI interaction also requires people to understand, correct, or reject AI output when necessary [7].

For Test Intent, this means that the representation should support several participants and tasks. A developer may need a short explanation of a test, while a reviewer or testing agent may need detailed activities

and links to supporting evidence. An agent could extract Test Intent and use it to propose new or improved tests. Software engineers and test engineers would still review the proposal and resolve unclear or conflicting information. In this workflow, the testing team would decide whether the proposed test and its stated intent should be accepted.

4 Data Scientists, Software Engineers, and AI Engineers

The chapters describe data scientists and software engineers as having different priorities. Data scientists tend to focus on data, models, experiments, and predictive performance. Their work often begins with an exploratory workflow and evaluates whether a model performs well on a given dataset. Software engineers are more concerned with the complete product. Their work includes requirements, architecture, implementation, testing, deployment, and maintenance [8]. I find this distinction useful, but I would not treat it as a strict division. It describes the typical focus of each role rather than a fixed boundary. A production system needs both perspectives, and the boundary depends on the organisation and the project.

The system-wide view is directly relevant to my research on Test Intent. It would be easy to treat the accuracy of Test Intent extraction as the main result. In practice, the extractor is only one part of a larger process. A usable system must locate the relevant test artifacts, preserve their context, validate the extracted structure, and present the result at a suitable level of detail. It must also update the intent when tests or related artifacts change. An extraction method may perform well in isolation but still provide little value if its output is outdated, difficult to inspect, or unsuitable for the downstream task. The evaluation should therefore consider not only extraction performance, but also whether the resulting Test Intent helps people understand, maintain, or generate tests. This follows the chapters' broader argument that model quality and system quality are related but not equivalent [8].

I expect data scientists and software engineers to remain distinct roles, although both will need some knowledge of the other's work. Building ML-enabled systems involves communication, documentation, engineering, and process challenges across disciplinary boundaries [9]. This does not mean that every team member must become an expert in both fields. Data scientists still provide specialised knowledge about data, modelling, and statistical evaluation. Software engineers provide expertise in system design, testing, integration, and operation. What matters is that each side understands enough of the other to discuss assumptions and recognise problems. My own research illustrates this need. Evaluating an LLM-based extractor requires knowledge of model behaviour and empirical evaluation, while deciding whether an extracted Test Intent is useful requires knowledge of software testing and development practice.

The term *AI Engineering* is less settled. The chapters note that it is often associated with model pipelines, deployment, and MLOps, which can leave it close to model engineering [8]. My view is that AI Engineering is partly distinct. It includes MLOps and model engineering, but its scope should also cover how an AI component interacts with requirements, users, software components, and operational constraints. An AI engineer does not replace either a data scientist or a software engineer. The role connects their work and pays particular attention to the behaviour of the complete AI-enabled system.

Foundation models and AI coding assistants make the boundary between experimentation and implementation less clear. Foundation models allow teams to build features through model selection, prompting, and evaluation instead of training every model from the beginning [8]. AI coding assistants also allow people to produce code and tests through an iterative interaction with a model, although the generated output still has to be understood and checked [6]. Prompt pipelines make this overlap more visible. Prompts, examples, output schemas, and model calls begin as experimental choices, but they become software artifacts once they are placed in a production system. They then need versioning, testing, and maintenance.

RAG systems bring data management, retrieval, generation, and software infrastructure into the same workflow [8]. Decisions about the document collection and retrieval method affect the model's output, while indexing, interfaces, latency, and monitoring are system concerns. Agentic workflows extend this interaction by allowing models to select tools and perform sequences of actions. Such workflows introduce questions about state, permissions, failure recovery, and the evidence used to guide an agent. Test

Intent could provide structured information about what a test is meant to verify when an agent generates or modifies tests. Its usefulness would depend on whether the workflow preserves the original evidence and checks the resulting actions. These technologies therefore increase the amount of work shared across roles, but they do not remove the need for specialised knowledge.

5 Analysis of Two Full Papers

5.1 RAGProbe: Scenario-Driven Evaluation of RAG Pipelines

The core idea of RAGProbe is to evaluate RAG pipelines through structured evaluation scenarios. An evaluation scenario specifies how documents and chunks are selected, how questions are generated, which prompting strategy is used, and how the answers are assessed. The framework contains a question–answer generator, a runner that submits the generated questions to a RAG pipeline, and a semantic evaluator that compares the returned answers with the expected answers. The authors define six scenarios covering numerical, temporal, multiple-choice, combined, cross-document, and unanswerable questions [10]. This approach is relevant to software engineering because it gives developers a repeatable way to generate and execute tests for RAG pipelines. The scenarios can also be extended when new failure patterns are identified.

The authors evaluate RAGProbe using five open-source RAG pipelines and three public datasets. Questions that combine several questions produce the highest failure rates, reaching 91%. The paper also reports that RAGProbe exposes more failures than the comparison approach [10]. I read these results as evidence that carefully designed scenarios can reveal weaknesses that are missed by more general question generation. They should not be interpreted as an estimate of how often these failures occur in actual use. The five pipelines have different models, embeddings, and default configurations, so the study does not provide a controlled ranking of them. Another limitation is that LLMs participate in both question generation and answer evaluation. An incorrect expected answer or judgement can therefore affect the reported result. The paper identifies further work on additional evaluation metrics, repeated execution, and the effect of different pipeline configurations.

RAGProbe can provide a useful reference for my research on Test Intent. An evaluation scenario describes how a family of test cases can be generated, whereas Test Intent explains what an individual test checks, why that check is needed, and how it is carried out. The two representations serve different purposes. However, RAGProbe shows why a structured description of test generation is useful when the number of possible inputs is too large for manual testing. Test Intent could complement such a description by making the purpose of each generated test easier to inspect. This connection is a possible research direction rather than something evaluated in my current study.

A suitable fictional project is an AI-assisted system for understanding and testing large software repositories. The system would use RAG to retrieve code, tests, requirements, issues, and review discussions. An LLM would then explain unfamiliar tests, extract their Test Intent, and suggest new or improved tests. RAG is a core part of this system because the required evidence may be distributed across several artifacts. RAGProbe's scenarios could provide reference cases for evaluating this retrieval and answering process. For example, a cross-document scenario could examine whether the system combines information from a requirement and its related test. An unanswerable scenario could examine whether the system reports that the available artifacts are insufficient, rather than generating an unsupported test purpose.

My research could contribute the Test Intent representation to this project. The Test Objects would identify the relevant code or behaviour. The Test Goal would state the verification purpose supported by the retrieved evidence. The Test Activities would describe how the test performs the check. RAGProbe's scenario structure could then inform how multiple test cases are generated for the same intent. This would connect generated tests to an explicit purpose instead of treating them only as executable code.

A possible long-term adjustment to my research is to record more information about the context in which an intent is extracted and used. This information could include links to the supporting artifacts, the

corpus version, the retrieval configuration, the prompt and model versions, and the evaluation criteria. Repeated extraction could be used to study whether an intent remains stable when these components change. Another study could compare tests generated from explicit Test Intent with tests generated from the same artifacts without that representation. Relevant measures would include the types of failures found, reproducibility, and human review effort. These extensions would preserve the current Test Intent structure while making it easier to study within a larger RAG-based testing workflow.

5.2 Just-in-Time Catching Test Generation at Meta

I selected this paper partly because Mark Harman is one of its authors and I met him at the ISSSE Summer School. Its topic is also close to my research. The paper examines a difficult question in automated testing: when a generated test fails on a proposed code change, does the failure reveal a defect, or does the test misunderstand what the developer intended to change? Becker et al. distinguish catching tests from conventional hardening tests. A hardening test passes when it is generated and protects the system against future regressions. A regression catching test instead passes on the parent revision but fails on the proposed revision. This difference establishes only a *weak catch*. It becomes a *strong catch* when the failure identifies an actual defect according to the expected behaviour of the system. Otherwise, it is a false positive caused by a mismatch between the test and the intended change. The main problem is therefore not simply generating a failing test, but determining whether the test should fail [11].

The paper evaluates two workflows that use information about a proposed code change. The *dodgy diff* workflow treats the change as though it were a mutant of its parent revision and generates tests that distinguish their behaviour. The *intent-aware* workflow first asks an LLM to infer how the intended change might go wrong. It represents these risks as mutants of the parent revision, generates tests that pass on the parent but fail on the mutants, and then runs the tests on the actual change. In the reported deployment, the workflow approximates intent from the changed code, its title, and its summary. Across 22,126 generated tests, the intent-aware workflow produced weak catches for 7.9% of the processed changes. The corresponding result was 4.0% for the dodgy diff workflow and 3.6% for the strongest non-diff-aware baseline. Rule-based and LLM-based assessors reduced the reported human review load by about 70%. Among 41 candidate catches reported to engineers, eight were confirmed as strong catches, and four could have caused serious production failures [11]. These figures provide evidence that the workflow is feasible at Meta, but they should not be treated as a general success rate. The engineer feedback covers a small set of reported cases, and the automated assessors show only modest agreement. I see the main contribution as the combination of automated generation, false-positive filtering, and human review, rather than any individual model or assessor.

For my research, the useful distinction is between *diff intent* and *Test Intent*. Diff intent concerns what a developer means to change. Test Intent describes what a test examines, why that behaviour matters, and how the test performs the check. A structured Test Intent could provide a test-level explanation alongside a candidate catch. It could identify the relevant test object, the behaviour being protected, and the activities and assertions that produce the failure. A reviewer could compare this explanation with the intended code change. Test Intent would not determine whether the change is correct, but it could make the source of a disagreement easier to identify. This idea could be explored in an AI-assisted pre-merge quality gate for a large online payment platform. A risk model could select changes that require additional testing. An LLM could infer possible implementation risks, while mutation-based and LLM-based generators produce candidate catching tests. Automated assessors could filter obvious false positives before the remaining cases reach the testing team. Each remaining test could be accompanied by a short Test Intent description, while an engineer would retain responsibility for deciding whether the observed change is expected.

The paper also suggests a possible change-aware extension to Test Intent. In addition to describing a test in isolation, the representation could record how its purpose relates to a parent revision and a proposed revision. It could distinguish behaviour that is expected to change from behaviour that should remain stable. Evidence for this distinction might come from an issue, a review discussion, or direct developer

input. A future study could compare generated tests presented alone with tests accompanied by this structured explanation. Relevant measures would include the proportion of candidates confirmed as strong catches, the time required for review, and agreement among independent reviewers.

6 Evidence, originality, and GenAI-use transparency

All main sources used in this report are listed in the References section. The course concepts came from the lecture and guest-lecture slides. I used these slides as course material rather than as peer-reviewed evidence. Chapters 1 and 2 of *Machine Learning in Production* are from an academic book. *RAGProbe* is a peer-reviewed CAIN paper, while *Just-in-Time Catching Test Generation at Meta* is a peer-reviewed FSE industry paper. The additional journal and conference papers support specific factual claims made in the report. The Test Intent manuscript is unpublished work and is used to describe my own research. I place the most weight on the original papers and book chapters because they present the relevant methods, evidence, and context directly. The lecture slides are mainly used to identify and discuss the concepts covered in the course.

I used GenAI in two limited ways. First, I used it as a discussion partner to examine some ideas and check whether my reasoning was clear. Second, after completing the initial draft, I used it to improve the English expression in the LaTeX text and correct grammatical errors. I reviewed the suggested changes and decided which ones to keep. GenAI was not treated as a source of evidence. I remain responsible for the analysis, factual claims, citations, and final wording.

References

[1] B. W. Boehm, "Verifying and validating software requirements and design specifications," *IEEE Softw.*, vol. 1, no. 1, pp. 75–88, 1984. [Online]. Available: https://doi.org/10.1109/MS.1984.233702

[2] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Trans. Software Eng.*, vol. 41, no. 5, pp. 507–525, 2015. [Online]. Available: https://doi.org/10.1109/TSE.2014.2372785

[3] R. Mohanani, I. Salman, B. Turhan, P. Rodríguez, and P. Ralph, "Cognitive biases in software engineering: A systematic mapping study," *IEEE Trans. Software Eng.*, vol. 46, no. 12, pp. 1318–1339, 2020. [Online]. Available: https://doi.org/10.1109/TSE.2018.2877759

[4] R. Artstein and M. Poesio, "Survey article: Inter-coder agreement for computational linguistics," *Computational Linguistics*, vol. 34, no. 4, pp. 555–596, 2008. [Online]. Available: https://aclanthology.org/J08-4004/

[5] B. Nuseibeh and S. M. Easterbrook, "Requirements engineering: a roadmap," in *22nd International Conference on on Software Engineering, Future of Software Engineering Track, ICSE 2000, Limerick Ireland, June 4-11, 2000*, A. Finkelstein, Ed. ACM, 2000, pp. 35–46. [Online]. Available: https://doi.org/10.1145/336512.336523

[6] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, pp. 85–111, 2023. [Online]. Available: [https://doi.org/10.1145/3586030

[7] S. Amershi, D. S. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. T. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, "Guidelines for human-ai interaction," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, CHI 2019, Glasgow, Scotland, UK, May 04-09, 2019*, S. A. Brewster, G. Fitzpatrick, A. L. Cox, and V. Kostakos, Eds. ACM, 2019, p. 3. [Online]. Available: https://doi.org/10.1145/3290605.3300233

[8] C. K"astner, *Machine Learning in Production: From Models to Products*. Cambridge, MA: MIT Press, 2025.

[9] N. Nahar, S. Zhou, G. A. Lewis, and C. K"astner, "Collaboration challenges in building ml-enabled systems: Communication, documentation, engineering, and process," in *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 2022, pp. 413–425. [Online]. Available: [https://doi.org/10.1145/3510003.3510209

[10] S. Sivasothy, S. Barnett, S. Kurniawan, Z. Rasool, and R. Vasa, "Ragprobe: Breaking RAG pipelines with evaluation scenarios," in *4th IEEE/ACM International Conference on AI Engineering - Software Engineering for AI, CAIN 2025, Ottawa, ON, Canada, April 27-28, 2025*. IEEE, 2025, pp. 60–71. [Online]. Available: https://doi.org/10.1109/CAIN66642.2025.00015

[11] M. Becker, Y. Chen, N. Cochran, P. Ghasemi, A. Gulati, M. Harman, Z. Haluza, M. Honarkhah, H. Robert, J. Liu, W. Liu, S. Thummala, X. Yang, R. Xin, and S. Zeng, "Just-in-time catching test generation at meta," in *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering, FSE Companion 2026, Concordia University, Montreal, QC, Canada, July 5-9, 2026*. ACM, 2026, pp. 282–293. [Online]. Available: https://doi.org/10.1145/3803437.3805202